Pokédex IoT

Informe de desarrollo y verificación de la Unidad 2

Versión 0.11.0 · 28 de octubre de 2026 · Proyecto individual

El proyecto conecta una aplicación Android con un servidor local para identificar Pokémon con la cámara, registrar los encuentros de cada usuario y compartir colecciones con amigos. La interfaz también permite monitorear y controlar pantalla, luces, audio, botones y volumen mediante un Arduino virtual.

La entrega incluye la APK, el código y evidencia de 56 pruebas automatizadas aprobadas. La demostración conjunta en dos teléfonos Android y la conexión de componentes físicos quedan por ejecutar. La pauta admite funcionalidades IoT simuladas y exige interconexión entre al menos dos Android mediante Wi-Fi o Bluetooth [1].

Problema y objetivo

Una captura aislada no permite mantener una colección personal ni compartirla con otro usuario. La solución reúne la lectura de un sensor, el procesamiento en un PC y la respuesta de actuadores en una experiencia sencilla de monitoreo y control. El objetivo de esta unidad es demostrar ese recorrido y conservar los datos de cada cuenta.

Arquitectura

Componente	Responsabilidad
Android A y Android B	La misma app con cuentas distintas. Cámara, fichas, controles y solicitudes de amistad.
Wi-Fi y HTTPS	Cada teléfono se comunica con la API del PC. El intercambio entre usuarios pasa por ese servidor.
Servidor Kotlin y SQLite	Autentica usuarios, conserva encuentros y controla el acceso a colecciones compartidas.
Ollama en el PC	Qwen3-VL 4B analiza un fotograma. El servidor limita el resultado al catálogo de 151 especies.
Arduino virtual	Aplica el estado enviado por la app y devuelve eventos de botones y perilla por el protocolo de dispositivo.

Se utiliza Kotlin con Jetpack Compose y Material 3 en Android; Kotlin/JVM y SQLite en el PC. Wi-Fi permite transportar imágenes sin ejecutar el modelo en el teléfono ni en Arduino. Bluetooth no es obligatorio para esta etapa [1].

Funcionamiento y control del dispositivo

Pantallas y recorrido

El usuario vincula el PC mediante QR y entra con usuario y contraseña. La colección muestra las especies vistas; las demás conservan silueta negra y datos ocultos. Al tocar Escanear se abre la cámara. Tras una identificación válida, la app registra el encuentro, cierra el visor y abre la ficha con efecto robótico y narración. Las fichas permiten deslizar entre especies y cambiar a una lista compacta.

Elemento	Monitoreo y activación
Cámara del celular	Sensor de imagen. El visor es continuo; se analiza un fotograma por consulta, sin acumular una cola.
Pantalla	Número, imagen y nombre de la ficha actual. Los datos de especies no vistas permanecen ocultos.
LED verde y amarillo	El verde refleja el estado controlable. El amarillo indica solicitudes de amistad pendientes.
Audio	Efecto y ficha hablada en el teléfono; activar, silenciar y ajustar volumen.
Botones y perilla	Anterior, siguiente, escanear, amigos, hablar, LED, silencio y volumen. Sus eventos regresan a la app.

Sincronización en ambos sentidos

La app conserva un estado solicitado y otro confirmado. Envía mensajes JSON con versión, sesión y número de secuencia; el dispositivo confirma mediante ACK. Un botón genera un evento que la app valida y convierte en un nuevo estado. Así se evita que cada pantalla mantenga valores diferentes.

Los ACK antiguos, eventos repetidos y mensajes de otra sesión no actualizan el estado. Si transcurren cinco segundos sin confirmación, se pierde el estado confirmado y se indica falta de enlace. Hay comprobaciones de actividad cada dos segundos cuando el enlace está inactivo. Al reconectar se transmite el último estado, sin repetir una narración anterior.

Las solicitudes entre cuentas se consultan cada cinco segundos con la app en primer plano. Aceptar comparte las colecciones; rechazar elimina la solicitud. La colección de un amigo no desbloquea la propia.

Preparación del Arduino físico

DeviceTransport, ArduinoChannel y FramedArduinoTransport definen la conexión y el ensamblado de mensajes. El documento hardware/PROTOCOL.md fija el contrato. Aún deben elegirse pantalla, audio, pines y transporte, y desarrollar los controladores y firmware correspondientes. Una imagen requiere una pantalla gráfica; un buzzer por sí solo no reproduce una ficha hablada.

Seguridad de la solución

Se toma ISO/IEC 27400 como referencia temática para riesgos, seguridad y privacidad IoT [2], e ISO/IEC 27001 para organizar la protección de información y el tratamiento de riesgos [3]. Esta correspondencia documenta medidas concretas del prototipo; no constituye certificación ni evaluación completa de esas normas.

Riesgo	Medida aplicada	Verificación y alcance
Suplantación del PC	HTTPS y huella SHA-256 del certificado fijada al vincular.	PcClientSecurityTest acepta la huella correcta y rechaza otra en TLS local.
Exposición de credenciales	Vinculación cifrada con AES-GCM y Android Keystore. Contraseña de usuario no guardada en el teléfono.	Revisión de ConnectionVault y LocalStore. Persistencia de Keystore pendiente en teléfono.
Robo de contraseña o sesión	PBKDF2-HMAC-SHA256 con sal y 210 000 iteraciones; tokens de sesión guardados como hash y con vencimiento.	DatabaseTest comprueba contraseña incorrecta, persistencia y revocación. Revisión del almacenamiento.
Acceso a datos ajenos	Identidad obtenida de la sesión. Colección compartida solo tras aceptación del destinatario.	AccountApiTest prueba identidad falsificada y aceptación por una tercera cuenta; operación rechazada.
Abuso de peticiones	10 intentos de autenticación por IP y minuto; cuerpos limitados; una inferencia a la vez.	AcceptanceTest verifica 429, 413 y JSON inválido. El límite de inferencia se revisa en código.
Filtración de imágenes	Fotogramas en memoria, catálogo local y sin guardado de fotos en el servidor.	Revisión de captura y Main.kt. No se ejecutó inferencia en esta validación.
Filtración por redirección	El cliente HTTPS no sigue redirecciones con sus credenciales.	PcClientSecurityTest verifica que una respuesta 302 no provoca acceso al destino.

Operación y riesgo residual

El PC es el punto central: su apagado interrumpe las cuentas y el reconocimiento. El QR contiene datos de vinculación y debe mostrarse solo a quienes participen. SQLite y el certificado permanecen en la carpeta privada del usuario, fuera del proyecto; la base no tiene cifrado propio en reposo. Deben protegerse el acceso al PC y sus respaldos. La autenticación del futuro enlace físico aún no está implementada.

Verificación de criterios OT

NIST SP 800-82 Rev. 3 orienta la seguridad OT considerando funcionamiento, fiabilidad y seguridad del entorno físico [4]. Aquí se aplican esos principios a un prototipo educativo. Las verificaciones lógicas del control no acreditan comportamiento eléctrico ni seguridad de hardware.

Criterio aplicado	Evidencia	Resultado y límite
Inventario y límites	Android, PC, SQLite, Ollama y dispositivo; arquitectura y protocolo versionado.	Componentes de software identificados. Modelo de módulos físicos pendiente.
Separación de servicios	API en la IP elegida; QR y Ollama usados por loopback. Android rechaza HTTP.	Revisión de Main.kt, PairingPage y manifiesto. La red del aula debe verificarse.
Estado confirmado	DeviceSyncTest: ACK tardío, desconexión y falta de confirmación.	Un envío no se presenta como aplicado hasta recibir el ACK correspondiente.
Integridad de comandos	DeviceSyncTest y ArduinoTransportTest: sesión, secuencia, duplicados, rangos, UTF-8 y tamaño.	Mensajes no válidos y eventos antiguos rechazados en las pruebas.
Pérdida de enlace	DeviceStabilityTest y AcceptanceTest: reconexión y reinicio de servicio.	Se recupera el estado y persisten datos. El watchdog local del hardware es trabajo futuro.
Recuperación de datos	AcceptanceTest copia una base temporal con VACUUM INTO, la abre y ejecuta quick_check.	Usuarios, sesiones, solicitud y encuentro se conservan; integridad comprobada.
Control de cambios	Versión 0.11.0, APK y hashes SHA-256 de fuentes; resultados JUnit conservados.	Permite relacionar esta entrega con la evidencia y repetir las verificaciones.

Procedimiento operativo

Antes de presentar, conectar PC y teléfonos a una red privada común; iniciar Start-Backend.ps1, vincular por QR y comprobar la conexión. No abrir puertos del router ni exponer Ollama. Si cambia la IP o el certificado, volver a vincular. Si falla el enlace con el dispositivo, reconectar y revisar el estado confirmado antes de continuar.

Para una copia manual de datos personales, detener primero el backend y copiar su carpeta privada a un destino protegido. Para restaurar, conservar una copia del estado actual, detener el servicio, recuperar los archivos elegidos y comprobar inicio de sesión y colección. La prueba automática de restauración usa exclusivamente una base temporal.

Resultados de las pruebas

La validación terminó correctamente: **56 casos aprobados**, 32 del módulo app y 24 del servidor, sin fallos ni errores. Se verificaron compilación de APK, pruebas JVM, Android Lint y compilación de pruebas instrumentadas. Estas últimas no se ejecutaron en Android.

Entorno: Windows, JDK 17 y Gradle 8.13. El intercambio de cuentas usa dos clientes JVM concurrentes y AccountApi con SQLite temporal sobre HTTP loopback. La prueba del cliente PcClient sí utiliza HTTPS y un certificado temporal. El protocolo Arduino se ejercita con su transporte virtual y fallos controlados. No se utilizó emulador, teléfono, Arduino físico ni inferencia de Qwen en esta ejecución.

Escenario	Evidencia obtenida
Dos usuarios concurrentes	200 escrituras correctas de 200; 100 especies por cuenta. Colecciones separadas y compartidas solo después de aceptar.
Tiempos del API local	Mediana 21.4 ms; p95 63.1 ms; máximo 280.0 ms. Duración 4.07 s; 49.1 peticiones/s.
Cambios bidireccionales	1000 ciclos; estado solicitado, aplicado y confirmado coinciden al final de cada ciclo. Duración 80.8 ms.
Reconexión del dispositivo	100 desconexiones y reconexiones conservan ficha, volumen y notificaciones sin repetir voz.
Caída y reinicio del API	Servicio detenido: conexión rechazada. Reinicio: sesión y encuentro recuperados. Repetir el registro no duplica especies.
Seguridad y entradas inválidas	Se rechazan sesión ausente/falsa, método incorrecto, JSON inválido, cuerpo excesivo, exceso de login y sesión revocada.
Transporte HTTPS	Certificado fijado aceptado; huella incorrecta rechazada; redirección no seguida.

Android Lint: 0 errores y 17 advertencias; el detalle está en lint-results-debug.xml. Los resultados JUnit, nombres de casos, métricas y hashes se incluyen en evidencia/. Los tiempos de loopback no miden Wi-Fi, Android ni velocidad de reconocimiento; el 95 % de las muestras queda en o por debajo del p95.

La confirmación previa del usuario de un reconocimiento exitoso en su celular es una observación de uso. No se utiliza como medición de precisión de las 151 especies ni como resultado de esta batería.

Correspondencia con la rúbrica

La tabla vincula cada criterio con una función y su evidencia. El puntaje corresponde al peso de la pauta, no a una calificación asignada. La evaluación final depende de la demostración y revisión del docente [1].

Criterio	Pts	Implementación y evidencia	Por demostrar
2.1.1.1 Login	10	Registro/login en Kotlin; cuentas SQLite. DatabaseTest y AccountApiTest.	Ingreso desde los dos Android.
2.1.1.2 Monitoreo y Activador	10	Material 3, ficha, LEDs, switches y volumen. DeviceSyncTest verifica ida y vuelta.	Uso visible de la interfaz en teléfonos.
2.1.2.3 Conectividad segura	15	HTTPS sobre Wi-Fi, QR y certificado fijado. Pruebas TLS y validación de configuración.	Enlace de ambos celulares a la red elegida.
2.1.2.4 Solución efectiva	10	Cámara como sensor, reconocimiento en PC y respuesta de actuadores. Recorrido implementado.	Reconocimiento y audio en la presentación.
2.1.3.5 Medidas ISO	10	Autenticación, cifrado, privacidad y control de acceso. Matriz de seguridad de este informe.	Verificación de Keystore y configuración de red en Android.
2.1.3.6 Lineamientos OT	10	Estados confirmados, control de mensajes, recuperación e inventario. Matriz OT y pruebas.	Integración eléctrica y watchdog cuando exista hardware.
2.1.4.7 Múltiples móviles	15	Misma app, cuentas distintas y amistad con consentimiento. Dos clientes concurrentes probados.	Prueba conjunta con dos dispositivos Android.
2.1.4.8 Estabilidad y eficiencia	20	Concurrencia, reinicio, 1.000 ciclos, 100 reconexiones, latencia local y rechazos comprobados.	Escenarios reales de Wi-Fi, permisos y ciclo de vida.

La simulación del dispositivo IoT forma parte del alcance permitido por las instrucciones de la unidad. La exigencia de demostrar la comunicación entre dos dispositivos Android sigue siendo independiente.

Guion de demostración y referencias

Demostración en clase

1. Iniciar el backend con .\Start-Backend.ps1 y mostrar el QR solo a los participantes. Instalar la APK en ambos Android y vincularlos al mismo PC.

2. Crear dos cuentas distintas. Explicar que SQLite está en el PC y que cada sesión identifica al usuario de las peticiones.

3. Abrir la colección: especies no vistas en negro. Escanear un Pokémon y mostrar la transición a ficha, voz y LED. Si se usa Prueba manual, indicar que se está registrando un dato de prueba.

4. En Ajustes > Dispositivo > Ver Arduino simulado, cambiar ficha, LED y volumen desde la app. Usar luego botones y perilla del panel y mostrar el cambio en la app.

5. Desde A enviar una solicitud a B. En B mostrar LED amarillo, aceptar y ver la colección compartida. Explicar la consulta cada cinco segundos en primer plano.

6. Desconectar y reconectar el dispositivo virtual. Mostrar estado sin confirmación y recuperación. Para red Wi-Fi, cortar y restablecer el enlace de un teléfono y registrar el resultado observado.

Explicación breve

“La cámara es el sensor. El PC procesa la imagen con Qwen y la app muestra una ficha del catálogo local. La API guarda usuarios y encuentros en SQLite. Dos usuarios se conectan al mismo servidor y comparten su colección con consentimiento. Para el dispositivo, cada cambio lleva una secuencia y se confirma; sus botones devuelven eventos a la aplicación.”

Repetir las pruebas

Ejecutar .\tools\Test-Delivery.ps1 desde PowerShell con JDK 17, Android SDK y Python disponibles. Se genera una carpeta nueva bajo artifacts/validation con log, XML, CSV, métricas y hashes. No se inicia el emulador ni se ejecuta Qwen. La primera compilación requiere acceso a las dependencias.

Referencias

[1] Instrucciones de Unidad 2 y lista de cotejo proporcionadas por el estudiante. Desarrollo Android en Kotlin, conectividad Wi-Fi o Bluetooth, dos dispositivos Android y control/monitoreo IoT.

[2] ISO/IEC 27400:2022. Cybersecurity — IoT security and privacy — Guidelines. iso.org/standard/44373.html. Consulta: 28-10-2026.

[3] ISO/IEC 27001:2022. Information security management systems. iso.org/standard/27001. Consulta: 28-10-2026.

[4] NIST SP 800-82 Rev. 3. Guide to Operational Technology Security, 2023. csrc.nist.gov/pubs/sp/800/82/r3/final. Consulta: 28-10-2026.

Datos e ilustraciones del catálogo: PokéAPI y PokeAPI/sprites, revisiones fijadas en data/kanto.json. Pokémon pertenece a sus titulares; proyecto educativo no oficial.